

Разбор задач чемпионата Student Fest 2026

Закарлюка, Плотников, Муравьев, Блинов

Санкт-Петербургский государственный университет

А. Проблемы со связью

Условие

- Одну строку отправили k раз. Номер каждой буквы мог измениться не более чем на d , без перехода по кругу.
- Известна сумма исходных букв C , где $a = 1, \dots, z = 26$.
- Найти лексикографически минимальную исходную строку.

Допустимые буквы

$t_{j,i}$ — номер буквы в позиции i копии j . Пересекаем ограничения всех копий:

$$L_i = \max\left(1, \max_j(t_{j,i} - d)\right),$$

$$R_i = \min\left(26, \min_j(t_{j,i} + d)\right).$$

Решение существует, если

$$L_i \leq R_i \quad \forall i, \quad \sum_i L_i \leq C \leq \sum_i R_i.$$

Заполняем справа налево

- Начинаем с $s_i = L_i$; остаток $\Delta = C - \sum_i L_i$.
- Справа налево добавляем $\min(\Delta, R_i - L_i)$ и уменьшаем остаток.
- Пока единицу можно перенести вправо, левая буква может стать меньше. Поэтому сначала заполняем правые позиции.

Пример: три интервала $[1, 3]$, сумма 6 $\rightarrow$ abc.

Время — $O(kn)$, память — $O(n)$.

В. Гномья бюрократия

Условие

- У листа $ID_v = C_v$, у внутренней вершины:

$$ID_v = \left(A_v \sum_{u \in \text{ch}(v)} ID_u + B_v \right) \bmod (10^9 + 7).$$

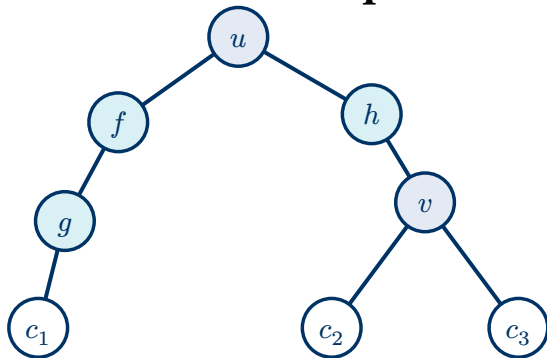
- После каждого изменения A_v , B_v или C_v нужен актуальный идентификатор корня.
- Среди любых 151 вершин есть предок и потомок.

Дерево почти состоит из цепочек

- Листья не являются предками друг друга. Значит, их число $L \leq 150$.
- Каждое ветвление увеличивает число будущих листьев; ветвлений не больше $L - 1$.
- Сжимаем вершины с одним ребёнком. Остаётся маленькое дерево из $O(L)$ листьев и ветвлений.

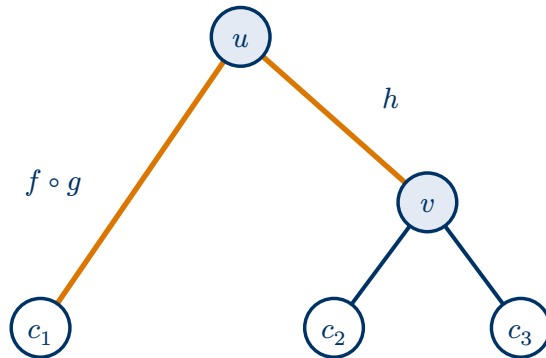
Сжимаем длинные цепочки

Исходное дерево



$\Rightarrow$

После сжатия



Голубые вершины заменяем композициями на рёбрах. Остаются 3 листа и 2 ветвления.

Цепочка — композиция функций

Вершина с одним ребёнком преобразует его значение:

$$f_{v(x)} = A_v x + B_v.$$

$$f(g(x)) = A_f A_g x + A_f B_g + B_f.$$

- Композиция снова имеет вид $ax + b$.
- Верхняя функция применяется к результату нижней: порядок композиции важен.
- Поддерживаем композицию деревом отрезков.

Обновление

1. Обновить функцию цепочки — $O(\log n)$.
2. Пересчитать сжатое дерево снизу вверх: применить функции рёбер и сложить вклады детей — $O(L)$.

Построение — $O(n)$, запрос — $O(\log n + L)$.

Длинные цепочки обрабатываем быстро; ветвлений мало.

С. Секретная пара

Условие

- Секрет — одна из t пар (X_i, Y_i) .
- Ваня знает X , Даня — Y . В каждом раунде одновременно отвечают, если уже знают секрет, или молчат.
- После общего молчания ведущий может исключить пару. Будущие подсказки неизвестны.
- Найти первый раунд ответа и победителя.

Пары как двудольный граф

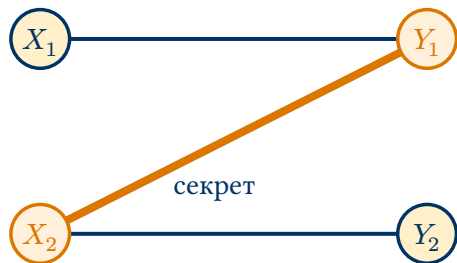
- Доли — значения X и Y , пары — рёбра.
- Степень 1 $\rightarrow$ с известной игроку координатой совместима ровно одна пара.
- Если секрет был бы при листе, кто-то ответил бы сразу. Общее молчание исключает такие рёбра.

Что сообщает общее молчание?

Раунд 1: оба молчат

Ваня: X

Даня: Y

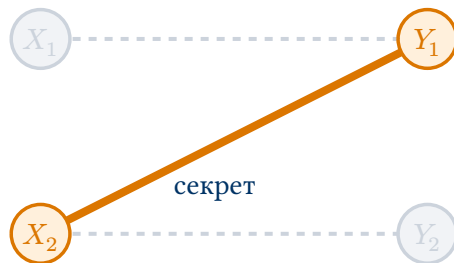


$\Rightarrow$

Раунд 2: оба знают

Ваня: X

Даня: Y



Секрет — оранжевое ребро. Сначала у его концов степень 2.
Молчание исключает пунктирные альтернативы; в следующем раунде оба знают ответ.

Один раунд — один слой

1. Проверить концы секретного ребра.
2. Удалить рёбра при листьях **начала раунда**.
3. Применить очередную подсказку.

Новые листья отвечают только в следующем раунде: каскад удалений нельзя обработать за один раунд.

Нет подсказок и новых листьев $\rightarrow$ NEVER.

Каждое ребро удаляем один раз: $O(m + q)$.

D. Организация чемпионата

Условие

- DAG зависимостей; каждая работа занимает день. Ребро $u \rightarrow v$ требует выполнить u строго раньше v .
- Число параллельных работ не ограничено.
- Для каждой работы: сколько дней ей доступны среди кратчайших расписаний?

Ранние и поздние дни

$$e_v = 1 + \max_{u \rightarrow v} e_u, \quad T = \max_v e_v.$$

$$h_v = 1 + \max_{v \rightarrow u} h_u, \quad l_v = T - h_v + 1.$$

- e_v — ранний день: длина пути **до** вершины в вершинах.
- После v нужны ещё $h_v - 1$ дней, поэтому поздний день $l_v = T - h_v + 1$.
- T — минимальная продолжительность всего расписания.
- Два прохода в топологическом порядке; пустой максимум — 0.

Между границами нет пропусков

Хотим поставить v в день $d \in [e_v, l_v]$.

$$\Delta = d - e_v, \quad c_u = \min(e_u + \Delta, l_u).$$

- Сдвигаем раннее расписание на Δ , затем ограничиваем поздними днями.
- Покоординатный минимум двух расписаний сохраняет зависимости.
- $c_v = d$, все работы укладываются в T дней.

Один граф — несколько расписаний

Зависимости



Нет зависимостей



Допустимые дни

Дни:	1	2	3
Работа 1			
Работа 2			
Работа 3			
Работа 4			

Цепочка фиксирует дни работ 1, 2, 3. Работа 4 может стоять в любом из этих дней. Ответ: 1 1 1 3.

Ответ

$$\text{answer}_v = l_v - e_v + 1 = T - h_v - e_v + 2.$$

Топологическая сортировка + две динамики.

Время и память — $O(n + m)$.

Е. Потеряшка

Условие

- Старт в $(0, 0)$; бесконечно повторяем строку команд L, R, U, D, двигаясь по одной клетке.
- Выход — первое касание $|X| + |Y| = x$.
- Найти точку выхода или сообщить, что его нет.

Из ромба в квадрат

$$u = X - Y, \quad v = X + Y.$$

$$|X| + |Y| = \max(|u|, |v|).$$

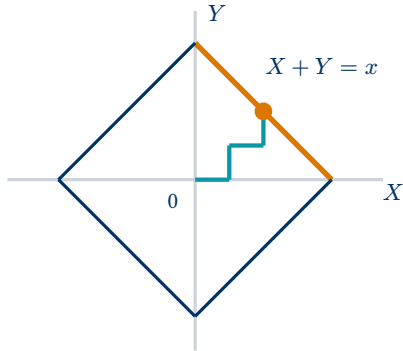
Внутри ромба:

$$-x < u < x, \quad -x < v < x.$$

Теперь достаточно отслеживать четыре границы двух координат.

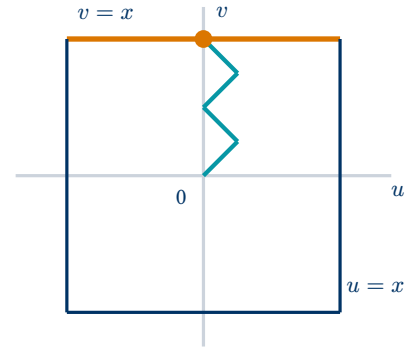
Та же траектория в новых координатах

Координаты X, Y



$\Rightarrow$

$u = X - Y, v = X + Y$



$x = 4$, путь RURU: выход $(2, 2) \rightarrow (0, 4)$. Оранжевая сторона $X + Y = x$ превращается в границу $v = x$.

Один проход описывает все повторения

- Сначала проверим выход на первом проходе.
- Для $z \in \{u, v\}$: экстремумы префиксов m_z , M_z и сдвиг δ_z .
- Каждый повтор — та же траектория со сдвигом. После k полных проходов диапазон:

$$[k\delta_z + m_z, k\delta_z + M_z].$$

(D_X, D_Y) — сдвиг за строку.

$$\delta_u = D_X - D_Y, \quad \delta_v = D_X + D_Y.$$

Пропускаем безопасные проходы

Индекс первого опасного прохода, считая с нуля:

$$K_z = \begin{cases} \left\lceil \frac{x - M_z}{\delta_z} \right\rceil \delta_z > 0 \\ \left\lceil \frac{x + m_z}{-\delta_z} \right\rceil \delta_z < 0 \\ \infty & \delta_z = 0. \end{cases}$$

- $K = \min(K_u, K_v)$. При $K = \infty$ выхода нет.
- Пропускаем K полных проходов. Из (KD_X, KD_Y) идём пошагово до первого касания стены.
- Итого $O(n)$.

Ғ. Поиcк cоcтавителей

Условие

- Составитель i : до x_i задач без напряжения, до y_i — всего.
 - Нужно ровно M задач.
 - Составитель напрягся, если получил больше x_i задач.
- Минимизировать число таких составителей.

Берём самые большие прибавки

- Бесплатная вместимость: $B = \sum_i x_i$.
- Напрягая составителя i , получаем ещё $c_i = y_i - x_i$ мест.
- Сортируем c_i по убыванию.
- Ответ — первый префикс с суммой $\geq M - B$.

Любые k составителей дают не больше, чем k наибольших прибавок. Последнюю можно использовать частично.

$M \leq B \rightarrow 0$; $M > \sum_i y_i \rightarrow$ невозможно. Время — $O(n \log n)$.

G. Разбиение на отрезки

Условие

- Разбить весь массив на подряд идущие непустые отрезки.
- Цена $[l, r] - \left\lfloor \frac{a_r}{a_l} \right\rfloor$.
- Максимизировать сумму; $n \leq 5 \cdot 10^5$.

Динамика и кандидаты

$\text{dp}[r]$ — лучший результат на первых r элементах.

$$\text{dp}[r] = \max_{0 \leq t < r} \left(\text{dp}[t] + \left\lfloor \frac{a_r}{a_{t+1}} \right\rfloor \right), \quad \text{dp}[0] = 0.$$

- Кандидат: $(a, b) = (a_{t+1}, \text{dp}[t])$.
- dp строго растёт: можно добавить одноэлементный отрезок.
- У нового кандидата больше b . При меньшем или равном a старый больше не нужен.
- В стеке оставляем возрастающие a и b .

Сравниваем прямые

Вклад кандидата $(a, b) - b + \lfloor \frac{z}{a} \rfloor$, где z — последний элемент отрезка. Так как b целое:

$$f(z) = b + \frac{z}{a}.$$

$$\max_t \lfloor f_{t(z)} \rfloor = \lfloor \max_t f_{t(z)} \rfloor.$$

- Максимум округлённых значений равен округлённому максимуму. Строим верхнюю огибающую **без округления**.
- Наклоны $\frac{1}{a}$ убывают.
- Каждая пара прямых имеет один порог переключения.

Пороги и стек

При $a_1 < a_2, b_1 < b_2$ новый кандидат лучше на малых z . Порог получаем из пересечения прямых:

$$T = \left\lfloor \frac{(b_2 - b_1)a_1a_2}{a_2 - a_1} \right\rfloor.$$

- При $z \leq T$ выбираем нового; при $z > T$ — старого.
- Пороги в стеке строго убывают; первый — ∞ .
- Новый порог не меньше последнего $\rightarrow$ у последнего не остаётся своего диапазона. Удаляем его и пересчитываем пересечение.

Запрос

1. Перед $\text{dp}[r]$ добавляем $(a_r, \text{dp}[r - 1])$.
2. Удаляем вытесненных кандидатов; восстанавливаем порядок порогов.
3. Бинпоиском находим последнего с $T \geq a_r$.
4. $\text{dp}[r] = b + \lfloor \frac{a_r}{a} \rfloor$.

Каждый кандидат входит и выходит из стека один раз.

Время — $O(n \log n)$, память — $O(n)$.

Н. Шахматные проказы

Условие

- Кодировем $x \leq 20\,000$ восемью не бьющими друг друга ладьями.
- Удаляют любые две ладьи.
- По оставшимся шести нужно восстановить x .

Два дополнения — разная чётность

Расстановка — перестановка столбцов по строкам.

$r \rightarrow a$	$r \rightarrow b$
$s \rightarrow b$	$s \rightarrow a$

Две пустые строки r, s ; два свободных столбца a, b .

Дополнения отличаются одним обменом $\rightarrow$ чётность числа инверсий различна. Ровно одно дополнение чётное.

Используем только чётные перестановки

$$\frac{8!}{2} = 20\,160 \geq 20\,000.$$

- Кодирование: x -я чётная перестановка в лексикографическом порядке.
- Декодирование: выбираем чётное дополнение и находим его номер. Это и есть исходная расстановка.
- Таблица одинакова в обоих запусках; перебираем всего $8! = 40\,320$ перестановок.

I. Экзамен

Условие

- Даны карточки с числами a_i . Игроки по очереди берут одну оставшуюся карточку и кладут в общую коробку.
- НОД стал 1 или нет хода $\rightarrow$ проигрыш.
- Выбрать сторону и выиграть; $a_i \leq n \leq 300$.

Состояние: число карточек и НОД

$\text{win}[i, g]$ — может ли выиграть следующий игрок, если уже взято i карточек с НОД $g > 1$.

- Все взятые карточки кратны g .
- Из исходных $\text{cnt}[g]$ кратных осталось $\text{cnt}[g] - i$.
- Некратные g точно ещё на столе; они уменьшают НОД.
Кратные сохраняют его.

Состав взятых карточек не важен.

Переходы

- Сохранить НОД: $(i, g) \rightarrow (i + 1, g)$, если $\text{cnt}[g] > i$.
- Уменьшить: $(i, g) \rightarrow (i + 1, h)$, где $h = \text{gcd}(g, a_j)$ и $1 < h < g$.
- Выигрышная позиция имеет переход в проигрышную. Ходы с новым НОД 1 сразу проигрывают.

База: $\text{win}[n, g] = \text{false}$. Считаем по убыванию i .

С таблицей НОД: $O(n^3)$ времени, $O(n^2)$ памяти.

Выбираем сторону и играем

- Есть $a_j > 1$ с проигрышным $\text{win}[1, a_j] \rightarrow \text{First}$.
- Иначе $\rightarrow \text{Second}$.
- Каждый ход оставляем сопернику проигрышное состояние: любой его безопасный ответ вернёт нас в выигрышное.
- Для восстановления хода перебираем оставшиеся карточки.

Ж. Тактический симулятор

Условие

- Для каждого подотрезка строим декартово дерево по минимуму.
- Выбираем максимум вершин без соседних по ребру — максимальное независимое множество.
- Отрезок стабилен, если корень входит хотя бы в один максимум.
- Посчитать стабильные отрезки; $n \leq 2 \cdot 10^5$.

Динамика независимого множества

$\text{dp}_1[u]$ — максимум с выбранным корнем, $\text{dp}_0[u]$ — без него. Дети вершины u — p и q .

$$\text{dp}_1[u] = 1 + \text{dp}_0[p] + \text{dp}_0[q],$$

$$\text{dp}_0[u] = \text{dp}[p] + \text{dp}[q],$$

$$\text{dp}[u] = \max(\text{dp}_0[u], \text{dp}_1[u]).$$

Введём $\Delta[u] = \text{dp}[u] - \text{dp}_0[u]$.

Вся информация — в одном бите

$$\Delta[u] = \max(0, 1 - \Delta[p] - \Delta[q]) \in \{0, 1\}.$$

- Пустое дерево: $\Delta = 0$.
- $\Delta[u] = 1 \iff$ у обоих детей $\Delta = 0$.
- **Нестабильность** $\iff$ у обоих детей $\Delta = 1$.

Если оба ребёнка дают прибавку, выбор корня теряет две вершины и добавляет лишь одну. Будем считать такие отрезки.

Зафиксируем минимум

- Полное декартово дерево строим монотонным стеком за $O(n)$.
- При фиксированном минимуме левая часть — суффикс левого поддерева, правая — префикс правого.
- S_u, P_u — числа непустых суффиксов и префиксов с $\Delta = 1$.

Каждый подходящий левый суффикс сочетается с каждым правым префиксом. Вклад вершины u :

$$S_{\text{left}(u)} \cdot P_{\text{right}(u)}.$$

Считаем суффиксы и префиксы

Дети — p слева и q справа. Начало суффикса:

- В q : вклад S_q .
- В u или в p : нужно $\Delta[q] = 0$. Слева подходит пустая часть или суффикс с $\Delta = 0$: всего $1 + \text{size}[p] - S_p$.

$$S_u = S_q + [\Delta[q] = 0] \cdot (\text{size}[p] + 1 - S_p),$$

$$P_u = P_p + [\Delta[p] = 0] \cdot (\text{size}[q] + 1 - P_q).$$

Для пустого дерева $\text{size} = S = P = \Delta = 0$.

Ответ

$$\text{answer} = \frac{n(n+1)}{2} - \sum_{u=1}^n S_{\text{left}(u)} P_{\text{right}(u)}.$$

- У каждого отрезка единственный минимум, поэтому вклады не пересекаются.
- Все величины считаем снизу вверх.
- Время и память — $O(n)$.